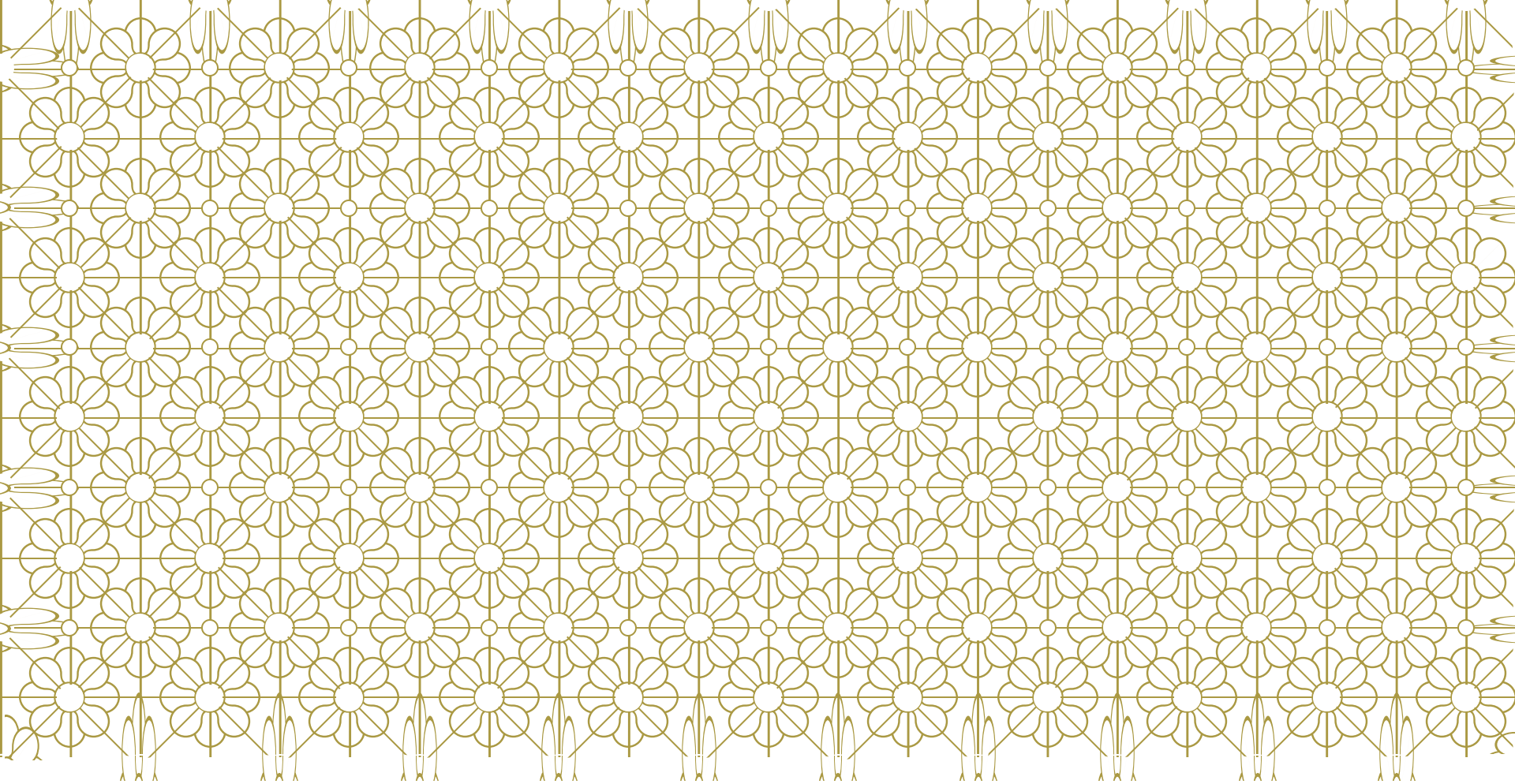
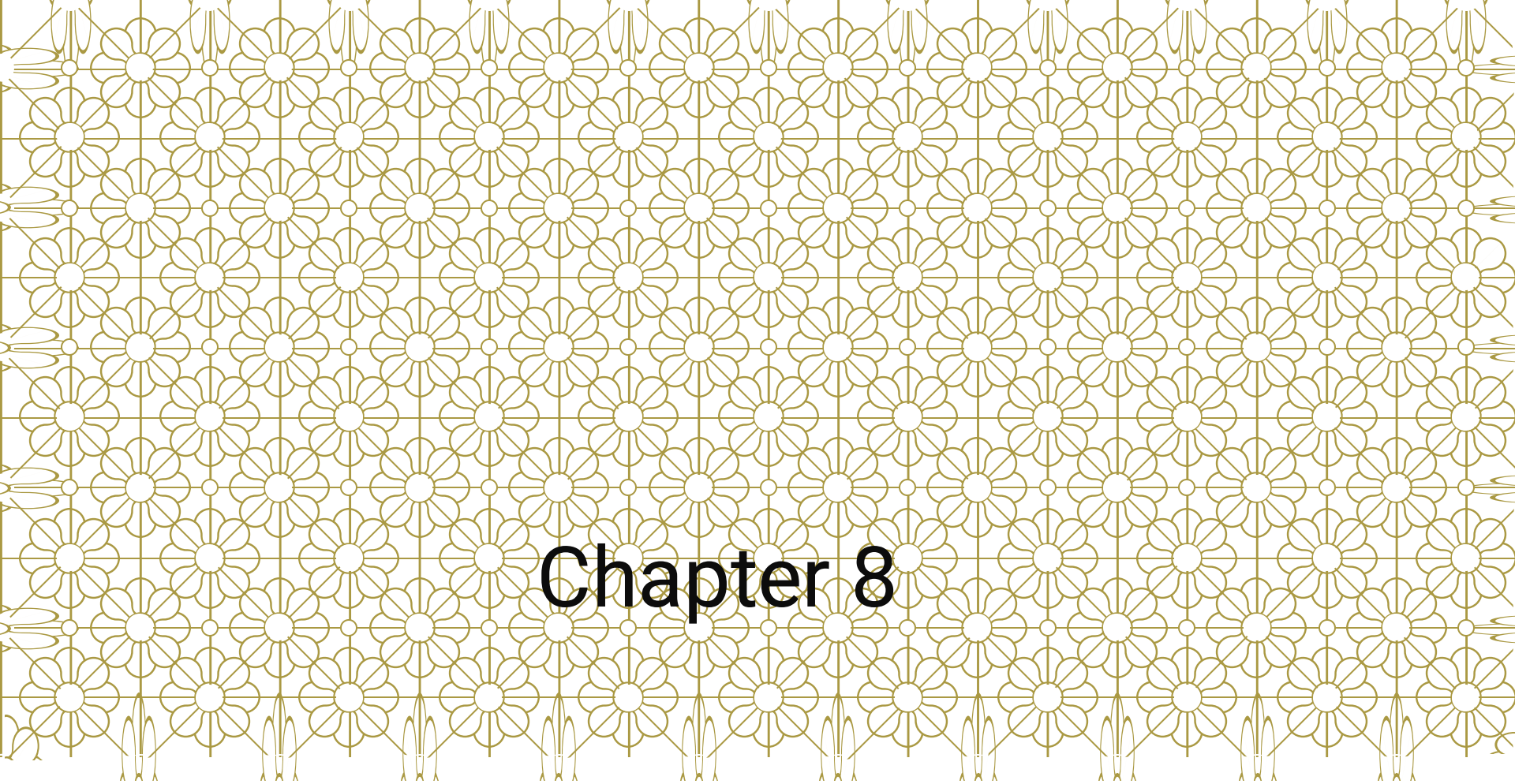




Algorithms analysis and Design

By : Dr. Basem
Alijla



Chapter 8

Dynamic Programming

OUTLINE

Introduction

Fibonacci Sequence using Dynamic Programming

Row Coin Problem

Dynamic Programming

- Dynamic Programming is a general algorithm design technique for solving problems defined by recurrences with overlapping subproblems
- Dynamic Programming is multistage decision process problem solving.
- Invented by American mathematician Richard Bellman in the 1950s to solve optimization problems and later assimilated by CS

Dynamic Programming

- “Programming” here means “planning” not “computer programming”.
- Dynamic programming is more efficient than brute-force
- Main idea:
 - set up a recurrence relating a solution to a larger instance to solutions of some smaller instances
 - solve smaller instances once
 - record solutions in a table

OUTLINE

Introduction

Fibonacci Sequence using Dynamic Programming

Row Coin Problem

Example 1: Fibonacci numbers

- “Fibonacci numbers (sequence) is a sequence in which the first two elements are 0, and 1 and all other elements are the sum of, the previous two elements.

$n = 0, 1, 2, 3, 4, 5, 6, 7, 8, 9$

$F(n) = 0, 1, 1, 2, 3, 5, 8, 13, 21, 34, \dots$

- Recall definition of Fibonacci numbers:

$$F(n) = F(n-1) + F(n-2)$$

$$F(0) = 0$$

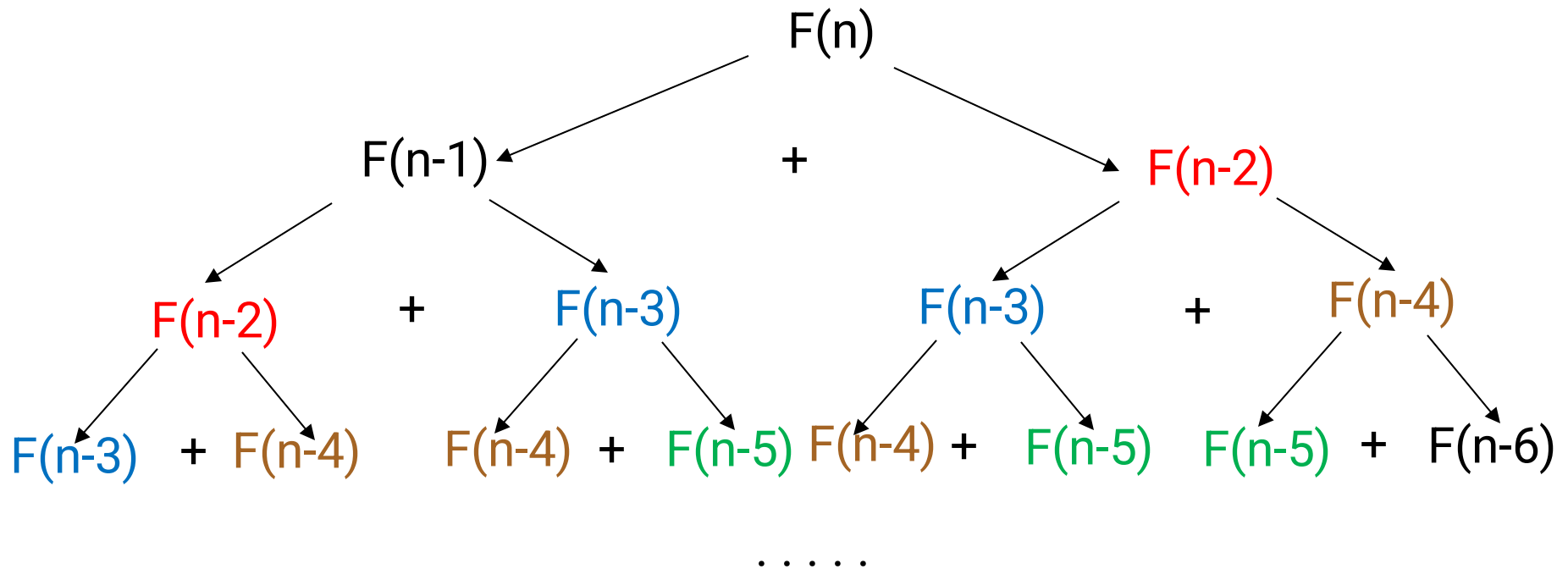
$$F(1) = 1$$

Example 1: Brute-force Fibonacci numbers

- Algorithm :
 - `Fib ( n ) {`
 - `if ( n < = 1 )`
 - `return n ;`
 - `else`
 - `return Fib(n-1) + Fib(n-2);`
 - `}`

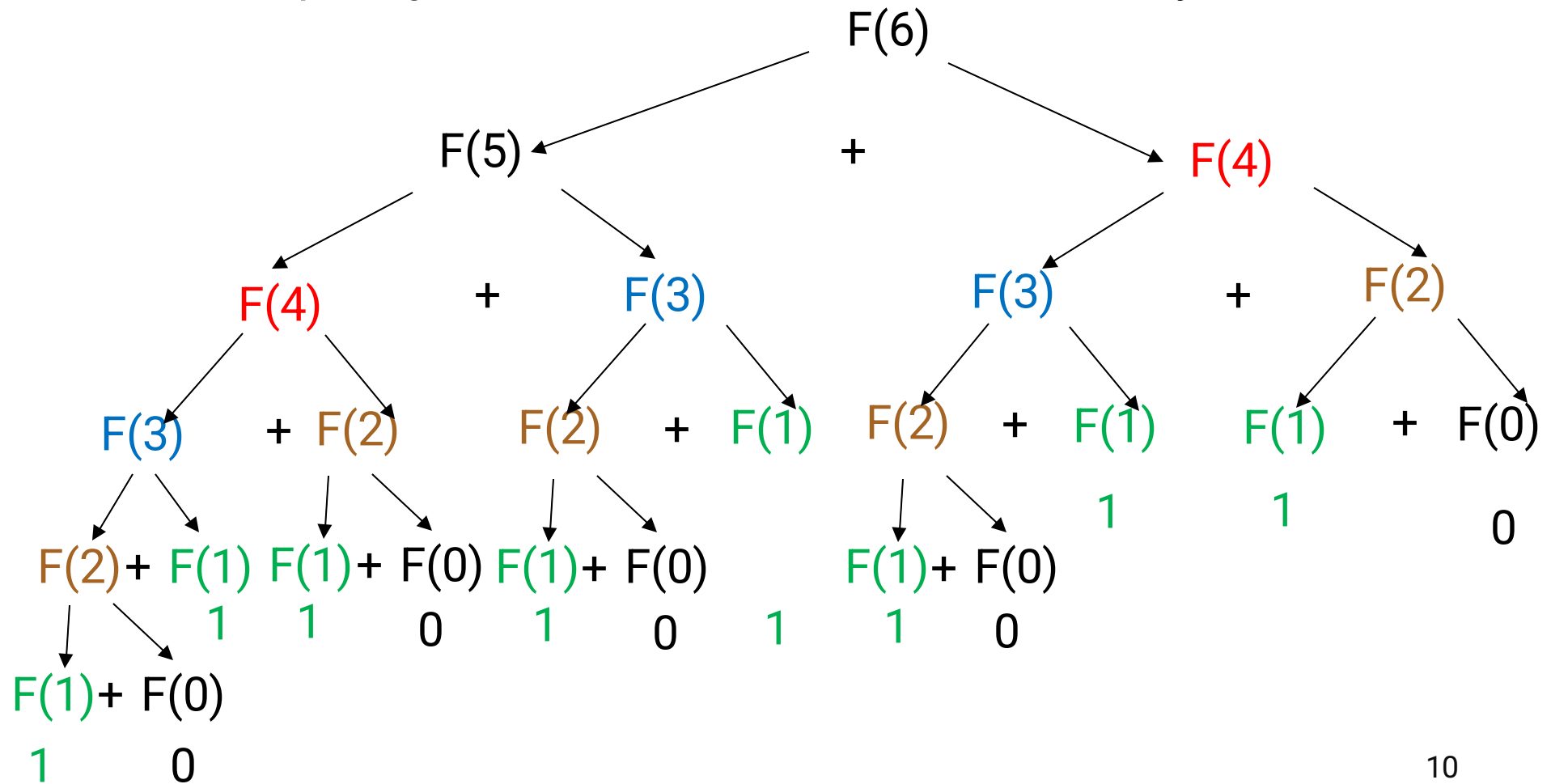
Example 1: Brute-force Fibonacci numbers

- Computing the nth Fibonacci number recursively :



Example 1: Brute-force Fibonacci numbers

- Computing the 6th Fibonacci number recursively:

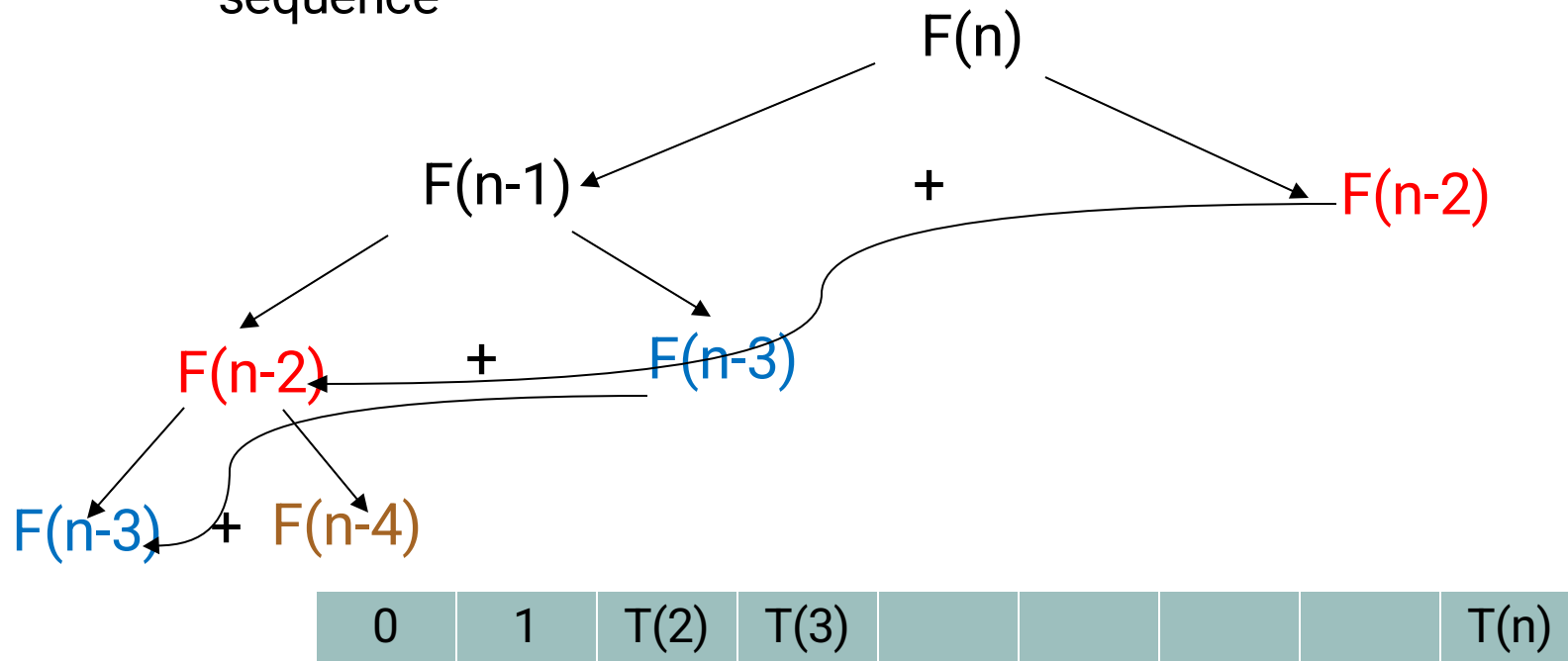


Efficiency analysis Brute-force Fibonacci

- $T(n) = T(n-1) + T(n-2) + c$
- $T(n-2) \approx T(n-1)$ in reality $T(n-2)$ take less than $T(n-1)$ Upper Bound
- $T(n) = 2T(n-1) + c$
- $T(n) = 2[2T(n-2) + c] + c$
- $T(n) = 4T(n-2) + 3c$
- $T(n) = 2^2 T(n-2) + (2^2 - 1)c$
- $T(n) = 2^3 T(n-3) + (2^3 - 1)c$
- $T(n) = 2^k T(n-k) + (2^k - 1)c$, $n-k = 0$, $k = n$
- $T(n) = 2^n T(0) + (2^n - 1)c$
- $T(n) = 2^n(1 + c) - c$ $O(2^n)$

Example 1: dynamic programming Fibonacci numbers (top-down)

- Computing the nth Fibonacci number recursively (top-down):
- Use a memory to store the previously calculated Fibonacci sequence



Example 1: dynamic programming Fibonacci numbers (top-down)

- Algorithm :
 - `FibTopDown ( n ) {`
 - `if(memo[n]>0)`
 - `return memo[n];`
 - `if ( n < = 1 )`
 - `result = n ;`
 - `else`
 - `result = FibTopDown(n-1) + FibTopDown(n-2)`
 - `memo[n] = result;`
 - `return result ;`
 - `}`

Efficiency analysis dynamic programming Fibonacci numbers (top-down)

- $T(n) = T(n-1) + T(n-2) + c$
- $T(n-2)$ is already calculated in the previous iteration
- $T(n) = T(n-1) + T(n-2) + T(n-3) + T(n-4) + \dots + T(0)$

Time $O(n)$

space $O(n)$

Efficiency analysis dynamic programming Fibonacci numbers (Bottom-up)

- Algorithm :

- `Fib ( n ) {`
- `memo[0] = 0 ;`
- `memo[1] = 1 ;`
- `for(int i = 2 ; i<= n ; i++)`
 - `memo[i] = memo[i-1] + memo[i-2]`
- `return memo[n]`
- `}`

Time $O(n)$

Efficiency analysis dynamic programming Fibonacci numbers (Bottom-up)

- Algorithm :
 - `Fib ( n ) {`
 - `a = 0 ;`
 - `b = 1 ;`
 - `c = a + b ;`
 - `for(int i = 2 ; i <= n ; i++){`
 - `c = a + b ;`
 - `a = b ;`
 - `b = c ;`
 - `}`
 - `return c ;`
 - `}`

Space $O(1)$

Approximated Fibonacci numbers by mathematical reduction

- $\phi = (1 + \sqrt{5}) / 2;$
- $\text{Fibo}(n) = \text{round}(\text{pow}(\phi, n) / \sqrt{5});$

Time $O(1)$
Space $O(1)$

principle of optimality

- principle of optimality an optimal solution to any instance of an optimization problem is composed of optimal solutions to its sub-instances.
- Sometimes it fail , e.g., it fails for finding the longest simple path in a graph
 -

OUTLINE

Introduction

Fibonacci Sequence using Dynamic Programming

Row Coin Problem

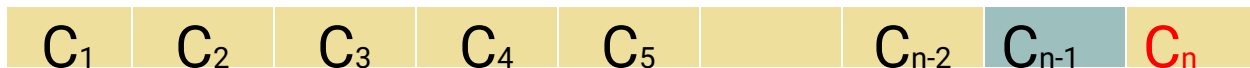
Example 2: Coin-row problem

- There is a row of n coins whose values are some positive integers $c_1, c_2, \dots, c_n$, not necessarily distinct.
- The goal is to pick up the maximum amount of money subject to the constraint that no two coins adjacent in the initial row can be picked up.
- E.g., 5, 1, 2, 10, 6, 2. What is the best selection?

•

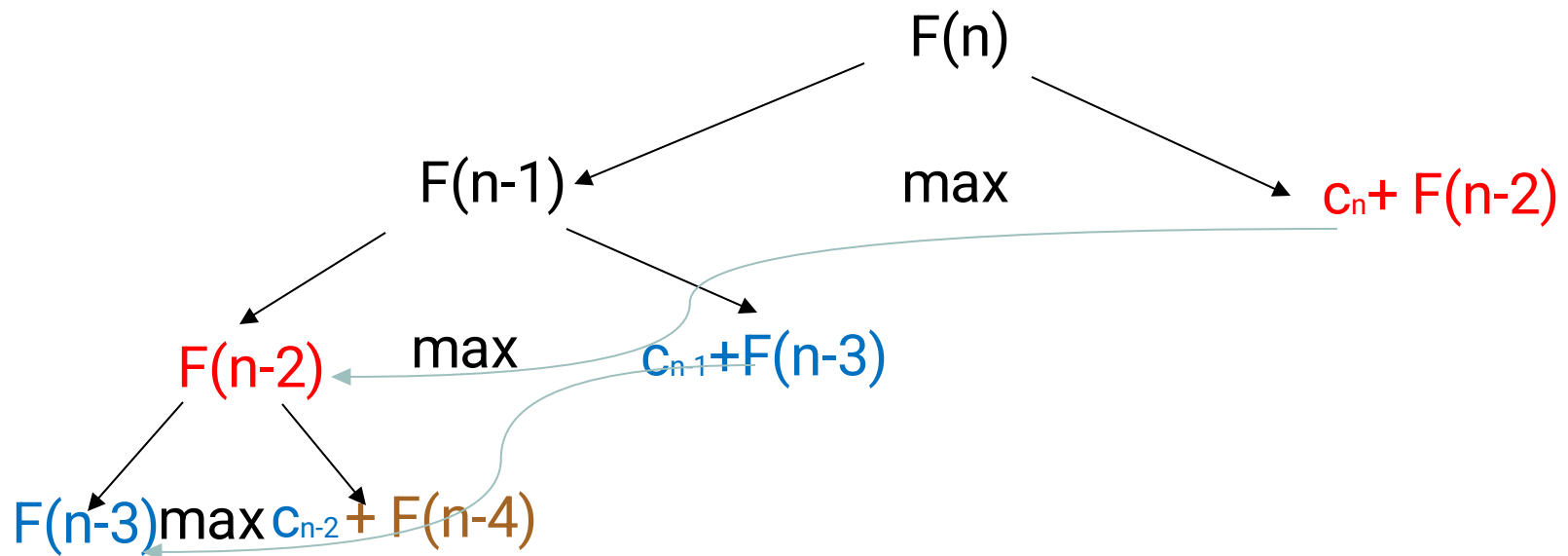
Example 2: Coin-row problem

- Let $F(n)$ be the maximum amount that can be picked up from the row of n coins.
- Partition the row of coins into two groups, those include the last coin and those without.



- $F(n) = \max \{ c_n + F(n-2), F(n-1) \}$
- $F(0) = 0, F(1) = c_1$

Example 2: Coin-row problem



Example 2: Coin-row problem- Algorithm

- Algorithm:
- $F[0] = 0$;
- $F[1] = c_1$;
- For(int i= 2, i <= n ; i++)
 - $F[i] = \max \{C[i]+F[i-2], F[i-1]\}$
- return $F[n]$

Example 2: Coin-row problem

- E.g.: 5, 1, 2, 10, 6, 2. What is the best selection?
- $F[0] = 0$, $F[1] = c_1 = 5$

index	0	1	2	3	4	5	6
C		5	1	2	10	6	2
F							

Example 2: Coin-row problem

- E.g.: 5, 1, 2, 10, 6, 2. What is the best selection?
- $F[0] = 0$, $F[1] = c_1 = 5$

index	0	1	2	3	4	5	6
C		5	1	2	10	6	2
F	0	5					

C_1

- $F[2] = \max \{c_2 + F[0] , F[1]\} = \max\{1+0, 5\} = 5$

Example 2: Coin-row problem

- E.g.: 5, 1, 2, 10, 6, 2. What is the best selection?
- $F[0] = 0$, $F[1] = c_1 = 5$

index	0	1	2	3	4	5	6
C		5	1	2	10	6	2
F	0	5	5				

C_1

C_1

- $F[3] = \max \{c_3 + F[1] , F[2]\} = \max\{2+5, 5\} = 7$

Example 2: Coin-row problem

- E.g.: 5, 1, 2, 10, 6, 2. What is the best selection?
- $F[0] = 0$, $F[1] = c_1 = 5$

index	0	1	2	3	4	5	6
C		5	1	2	10	6	2
F	0	5	5	7			

- $F[4] = \max \{c_4 + F[2] , F[3]\} = \max\{10 + \overset{C_1 + C_3}{5}, 7\} = 15$

Example 2: Coin-row problem

- E.g.: 5, 1, 2, 10, 6, 2. What is the best selection?
- $F[0] = 0$, $F[1] = c_1 = 5$

index	0	1	2	3	4	5	6
C		5	1	2	10	6	2
F	0	5	5	7	15		

- $F[5] = \max \{c_5 + F[3] , F[4]\} = \max\{6 + \overset{C_1 + C_4}{7}, 15\} = 15$

Example 2: Coin-row problem

- E.g.: 5, 1, 2, 10, 6, 2. What is the best selection?
- $F[0] = 0$, $F[1] = c_1 = 5$

index	0	1	2	3	4	5	6
C		5	1	2	10	6	2
F	0	5	5	7	15	15	

- $F[6] = \max \{c_6 + F[4] , F[5]\} = \max\{2+15, 15\} = 17$

Example 2: Coin-row problem

- E.g.: 5, 1, 2, 10, 6, 2. What is the best selection?
- $F[0] = 0$, $F[1] = c_1 = 5$

index	0	1	2	3	4	5	6
C		5	1	2	10	6	2
F	0	5	5	7	15	15	17

- Back-trace $\{ C_n + F[n-2], F[n-1] \}$ that produce the maximum to determine the selected coins $C_1 + C_4 + C_6$
- $F[6] = \max \{ c_6 + F[4], F[5] \}$ $c_6 + F[4]$ then c_6 is part of the solution
- $F[4] = \max \{ c_4 + F[2], F[3] \}$ $c_4 + F[2]$ then c_4 is part of the solution
- $F[2] = \max \{ c_2 + F[0], F[1] \}$ $F[1]$ then c_1 is part of the solution

Performance Analysis Coin-row algorithm

- Algorithm:
- $F[0] = 0$;
- $F[1] = c_1$;
- For(int i= 2, i <= n ; i++)
 - $F[i] = \max \{C[i]+F[i-2], F[i-1]\}$
- return $F[n]$

$O(n)$